

Natural Computation Methods for Machine Learning

1DL073 61011 VT2025

Project Report

Project Supervisor

Olle Gällmo

Group - 14

Eric Dario Márquez León

Thant Zin Bo

Moutushi Sen

Date of Submission: June 05 2025



UPPSALA
UNIVERSITET

Genetic Algorithm for Feature Selection in Semiconductor Manufacturing Fault Detection

Abstract

This project investigates the application of genetic algorithms (GAs) for the selection of subsets of features on synthetic and real-world datasets. GA performance is evaluated using the publicly available semiconductor manufacturing process (SECOM) dataset by comparing classification metrics against the baseline balanced error rate (BER) provided with the dataset. In addition, a controlled synthetic dataset based on SECOM is constructed and evaluated using the same GA configuration. The industrial motivation is to optimize the sensor input space by reducing the dimensionality, thus improving the accuracy of fault detection and reducing monitoring costs. Experiments on both datasets highlight the importance of selecting a sufficiently large population, the complexity of creating a synthetic dataset and demonstrate the potential of GA-based wrapper methods for industrial quality assurance and future multiobjective optimization research.

1 Introduction

Modern manufacturing facilities generate large amounts of sensor data that pose tremendous analysis challenges; some of these include features that could be redundant or mask significant signals, high computational complexity that strains resources, and escalating maintenance costs for the monitoring system [8]. To address these difficulties, in this research, genetic algorithms are applied within a feature selection approach. This approach is used to determine the optimal set of values that improve the performance of the defect detection process and minimize complexity. The approach employs a two-step strategy to verify the results. Firstly, it demonstrates the algorithm’s performance on the semiconductor manufacturing (SECOM) dataset. Secondly, it validates the algorithm using synthetic data that preserve the critical characteristics of the real manufacturing dataset. The two-step verification serves to ensure the accuracy and applicability of the method for the development of improved process monitoring systems.

2 Theory

Feature selection is a key step in building efficient machine learning models. By identifying a minimal subset of relevant features, models can achieve better generalization, lower complexity, and faster inference. The primary purpose of feature selection is to design a more compact classifier with as little performance degradation as possible. The features removed should be redundant or of the least possible use.

2.1 Genetic Algorithm-Based Feature Selection

Genetic Algorithms (GAs) borrow the principles of natural genetics and natural selection and apply them to construct robust search algorithms that require minimal problem information. The goal here is to select a subset d of features from a set of D features based on some optimization criterion, hence decreasing the size of the search space from 2^D to 2^d . In this research, the following notation is defined as in:

The D features are uniquely identified by distinct numbers from 1 to D , so the total set is written as:

$$U = \{1, 2, \dots, D\}.$$

X denotes the subset of selected features and Y the set of remaining features, i.e.,

$$U = X \cup Y$$

at any time.

$J(X)$ denotes a function evaluating the performance of X , which, according to, it may evaluate either the accuracy of a specific classifier on a specific data set called the wrapper approach or a generic statistical measurement, called the filter approach; the actual choice depending on the particular application. In this research, a wrapper approach is proposed because this measures how well the classifier performs with the given feature subset X .

2.2 Working Principle

Genetic Algorithms are naturally applicable to feature selection since the problem has an exponential search space (2^D possible subsets). Exhaustive search becomes infeasible for large D , making GAs a suitable heuristic search strategy. Through their algorithm's iterations, called generations, they maintain a set of solutions called individuals or chromosomes in a population. As in the case of biological evolution, they have a mechanism of selecting fitter chromosomes at each generation. A major aspect to be defined to find a solution in a large search space is how to control the genetic operation, i.e., a method to control the population. There are two alternatives: Steady state procedure, in which a chromosome is updated in each generation. Generational procedure: where the entire (or most of) the population is updated [6]. A flow chart of a GA can be seen in Figure 1.

2.3 Solution Representation

Binary chromosome encoding is a natural choice for representing feature selection problems by aligning each feature directly with a gene. For a feature reduction problem, a binary string with D binary digits is created. Each digit represents the feature that matches its position within the string; a value of 1 means the respective feature has been selected, whereas a value of 0 indicates that such feature has been removed.

For example, a chromosome representing $D = 8$ features with values: 01100100 means that the second, third, and sixth features have been selected, leaving the remaining features unselected. This is expressed as: $X = 2, 3, 6$ and $Y = 1, 4, 5, 7, 8$.

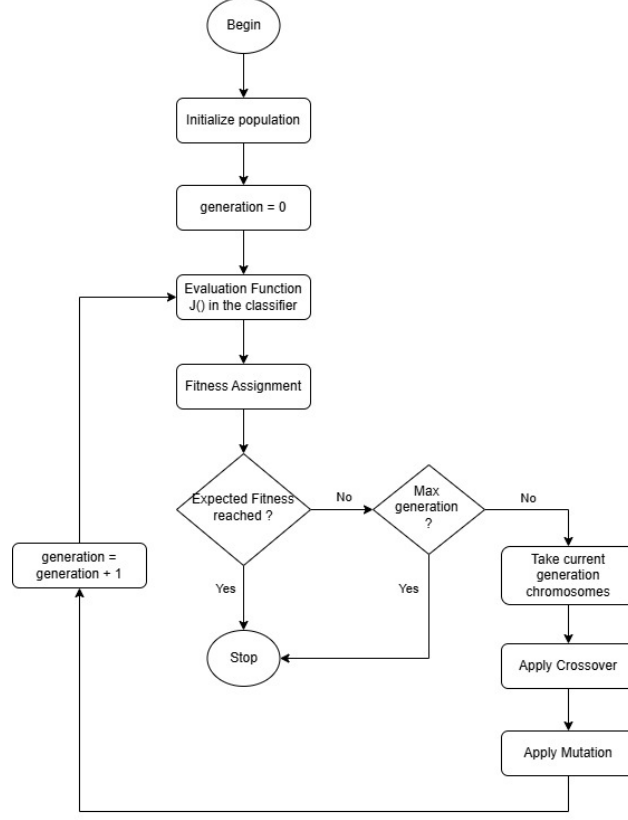


Figure 1: Flowchart of the working principle of the proposed GA, inspired by the model presented.

2.4 Population initialization

As stated before, the expected number of selected features in any solution is d , where $d \subseteq D$; this constraint must be set up since the first population makes it more likely for the algorithm to focus on small subsets instead of larger ones in further generations. A probability of $\frac{d}{D}$ ensures that, on average, each chromosome selects around d features initially, promoting compact feature subsets from the start, that is, setting the chance of each gene being 1 in the initial population P to a value of $\frac{d}{D}$.

$$p_1 \leq \frac{d}{D}; \quad p_0 = 1 - p_1$$

2.5 Fitness assignment

To enforce the constraint of a size value d , a penalty is imposed on chromosomes breaking it, and the fitness of a chromosome C can be defined as:

$$\text{Fitness}(C) = J(X_C) - w \cdot ||X_C| - d|$$

where X_C denotes the number of features selected in the chromosome C . The penalty discourages solutions that are significantly larger or smaller than the target subset size and

is defined with a penalty coefficient w as:

$$\text{Penalty} = w \cdot ||X_C| - d|$$

2.6 Selection method of the most fit individuals

Chromosome selection in each generation is done through the evaluation of the fitness function, making fitter individuals more likely to survive and pass their genes to future generations. However, maintaining only the best-fitted individuals in the population may lead to premature convergence to local optima, potentially overlooking better individuals in future generations. It is generally advisable to maintain diversity in the population. To this end, a rank-based roulette-wheel selection mechanism can be chosen: better-ranked individuals would have higher probabilities of selection by a nonlinear function, but worse individuals would still have a small chance as well. The chromosomes are ordered by their fitness values in descending order and assigned a probability according to the formula:

$$P(\text{selection}) = \frac{n - \text{rank} + 1}{\sum_{i=1}^n i}$$

where n is the total number of chromosomes in the population and rank is the rank of the chromosome based on its fitness.

2.7 Next generation definition

The next generation will be obtained by replacing totally (generational replacement strategy) or partially (steady state replacement strategy) the individuals in the current generation. For example, in steady state, when two chromosomes are selected following the method above, they will generate a new chromosome (offspring) through a crossover operation, which could also be slightly changed by the mutation operation. If the offspring has higher fitness than both parents, it replaces the similar parent; if its fitness lies between the two parents, it replaces the inferior parent; otherwise, the most fit chromosome in the population is replaced. The GA is terminated once either a predefined number of generations T is reached or a solution achieving the target fitness is found. In addition, elitism could be applied to preserve the best individuals of each generation, that is, the top e individuals are copied to the next generation.

2.7.1 Crossover Operator

In feature selection problems, uniform crossover is often recommended because the positional order of features is arbitrary, and maximizing the random combination of genes can help the algorithm explore the search space more effectively. In this project, a uniform n-point crossover operator is applied.

2.7.2 Mutation

Following crossover, a mutation operator is applied with low probability to individual genes to introduce small random variations, further enhancing diversity. The mutation of each

gene is controlled by the formula:

$$p_0 = p_1 \cdot \frac{n_1}{n_0}$$

where p_1 is the mutation rate from 1 to 0 and p_0 is the mutation rate from 0 to 1. Scaling p_0 using the ratio $\frac{n_1}{n_0}$ ensures that, on average, the number of genes flipping from $1 \rightarrow 0$ and from $0 \rightarrow 1$ is similar. Thus, the total number of selected items (1s) stays roughly constant, even after mutation.

2.8 Constraints Enforcement Mechanism

It is important to note that after applying crossover and mutation, a repair mechanism may be required to ensure that the offspring satisfies the desired subset size constraint. The choice of crossover, mutation, and constraint-handling strategies must ensure the exploration of the search space while biasing the search toward compact, high-performing feature subsets. As a result, GAs provide a flexible and powerful approach to feature selection, particularly suited to problems with large, complex search spaces.

3 The SECOM Dataset

3.1 Exploratory Data Analysis on SECOM Dataset

The semiconductor manufacturing process (SECOM) dataset [4] is a cross-sectional collection of semiconductor manufacturing sensor readings paired with pass/fail outcomes. It consists of 1,567 product-test instances, each described by 591 real-valued features extracted from process sensors, along with a binary yield label and a timestamp for each test.

It has missing values in 538 features ($\approx 4.5\%$ of all entries) as well as 1,463 passes (93.4%) and 104 fails (6.6%), and it is then considered a highly imbalanced dataset. This imbalance is expected since it emanates from an industrial manufacturing process; therefore, defective products are expected to be very rare. This means that most of the data points (signals) correspond to the non-defective class, which are considered as the positive class.

SECOM dataset has features far exceeding typical modeling dimensions, leading to multicollinearity and the need for dimensionality reduction (e.g., PCA) or feature selection. Many features exhibit strong linear and non-linear relationships, necessitating generation methods that preserve joint dependencies.

The Figure 2, PCA on all features shows heavy overlap of pass/fail classes in the first two components, indicating no simple linear discriminant dataset is a traditional time series, nor is it continuous. Each row is a single snapshot of 591 features at one timestamp. There is no regular temporal sampling within each feature stream. Aggregating by test date reveals trends in daily failure rates; hourly histograms expose shift-based failure patterns.

Based on Figure 3, distribution is right-skewed (positively skewed), with a long tail extending to the right. The bulk of the data is concentrated between 18 and 24 Mahalanobis distance units. There is a steep drop in frequency after around 25. A handful of samples lie far to the right on PC1 (values up to ≈ 100) or high on PC2 (up to ≈ 30)

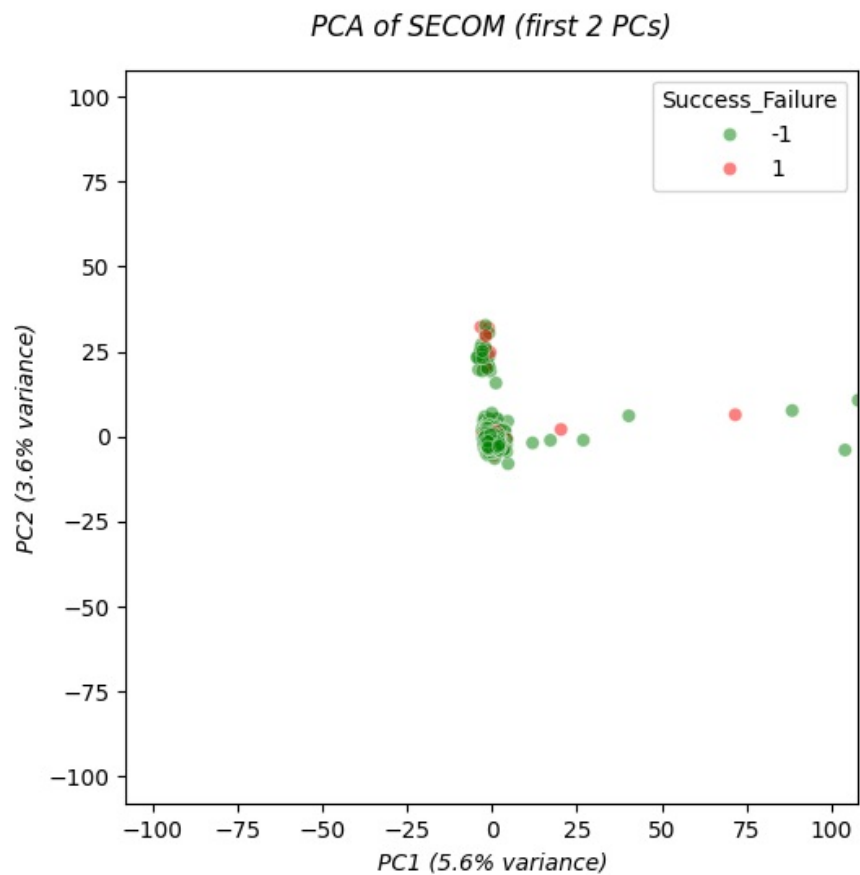


Figure 2: PCA graph of SECOM Dataset

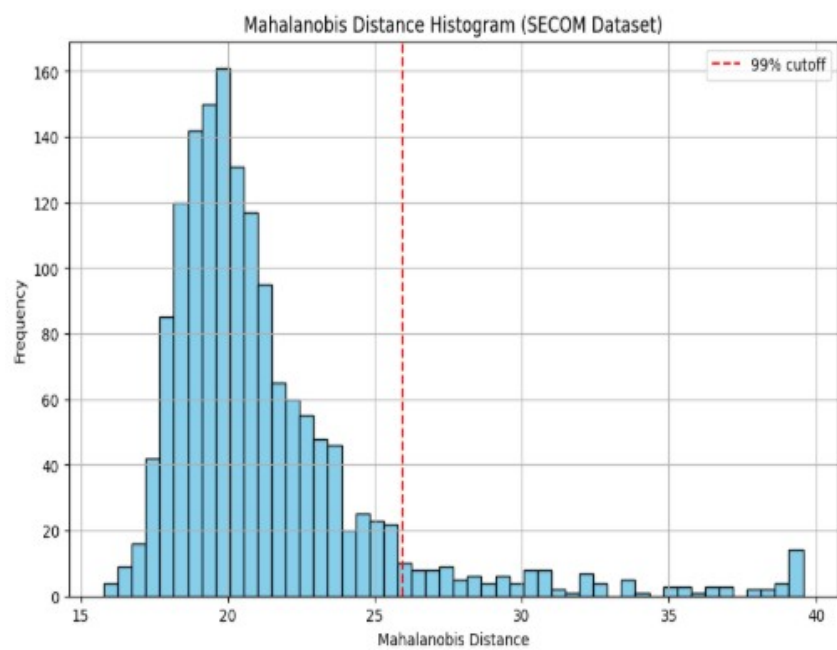


Figure 3: Mahalanobis distance histogram

3.2 Baseline Results

As stated in the SECOM documentation, pre-processing objects were applied to the dataset to standardize the data and remove the constant features. Then, a number of different feature selection objects selecting the 40 highest ranked features were applied with a simple classifier to achieve some initial results. 10-fold cross-validation was used, and the balanced error rate (BER) was generated as the benchmark initial performance metric to investigate this dataset. A comparison of the results of different feature selection methods (FS) can be seen in Table 1.

- **True + %:** Corresponds to the True Positive Rate (sensitivity), which is the percentage of actual positives correctly predicted (higher is better).
- **True - %:** Corresponds to the True Negative Rate (specificity), which is the percentage of actual negatives correctly predicted (higher is better).
- **The +- values:** Represent standard deviation or variability over repeated experiments (e.g., cross-validation folds).

FS Method (40 features)	BER %	True + %	True - %
S2N (Signal to noise)	34.5 $\pm$ 2.6	57.8 $\pm$ 5.3	73.1 $\pm$ 2.1
Ttest	33.7 $\pm$ 2.1	59.6 $\pm$ 4.7	73.0 $\pm$ 1.8
Relief	40.1 $\pm$ 2.8	48.3 $\pm$ 5.9	71.6 $\pm$ 3.2
Pearson	34.1 $\pm$ 2.0	57.4 $\pm$ 4.3	74.4 $\pm$ 4.9
Ftest	33.5 $\pm$ 2.2	59.1 $\pm$ 4.8	73.8 $\pm$ 1.8
Gram Schmidt	35.6 $\pm$ 2.4	51.2 $\pm$ 11.8	77.5 $\pm$ 2.3

Table 1: Balanced Error Rate for the 40 highest ranked features in the SECOM dataset

4 Experiments with the SECOM Dataset

The method begins with heavy preprocessing of data, including imputation of missing values and feature standardization. A genetic algorithm (GA) is then employed, whose key parameters (population size, crossover/mutation rates) are grid-search-optimized to obtain an exploration-exploitation trade-off. Performance is measured in terms of classification metrics (accuracy, precision, recall, F1-score) and efficiency indicators (subset size, runtime). Statistical significance is verified by paired t-tests or Wilcoxon signed-rank tests against baseline methods, ensuring rigorous and impartial testing of the described feature selection approach.

4.1 Early experimentation

4.1.1 Data Preprocessing

The early experiments were done without eliminating or standardizing any features, The idea was to see how well the algorithm could perform with a minimum preprocessing of the

data set, which is: nothing beyond standard cleaning steps, like conversion to numeric and missing values imputation was applied.

4.1.2 Splitting of the dataset for Fitness Evaluation

Despite the known imbalanced nature of the dataset, the splitting was done without the intention of maintaining the proportion of class labels; 70% was assigned for training, and 30% was kept for further validation of the classifier model.

Name	Value / Type	Description
Initial population	Random	To make it more likely for the algorithm to focus on small subsets instead of larger ones in further generations.
Selection Mechanism	Rank-based with selection coefficient from 0.4 to 0.7	To select pairs of parents using rank-based roulette wheel selection
Genetic operation control	Generational	Updates the whole or most of the population with the offspring
Crossover	Uniform from 1 to 3 points. Rate = 1	
Mutation	Rate from 0.01 to 0.1	
Population Size	50	
Number of generations	100	
Penalty coefficient	From 0.001 to 0.1	Higher value means GA can explore farther from the desired <i>features number</i> .
Target Features Number	50	Based on the benchmark

Table 2: Experiment Parameters

4.1.3 Fitness function

The underlying classifier model is a Random Forest Classifier with hyperparameters set to default values. The classifier score is calculated using the `f1_score` as a metric with a binary type of average. This is expected to be a more suitable performance metric for this dataset. The fitness score is calculated as as seen here:

$$\text{score} = \text{f1_score}(y_{\text{val}}, y_{\text{pred}}, \text{average} = \text{'binary'}, \text{pos_label} = 1)$$

$$\text{penalty} = \text{penalty_coef} \times |\text{num_selected_features} - \text{num_desired_features}|$$

$$\text{fitness} = \text{score} - \text{penalty}$$

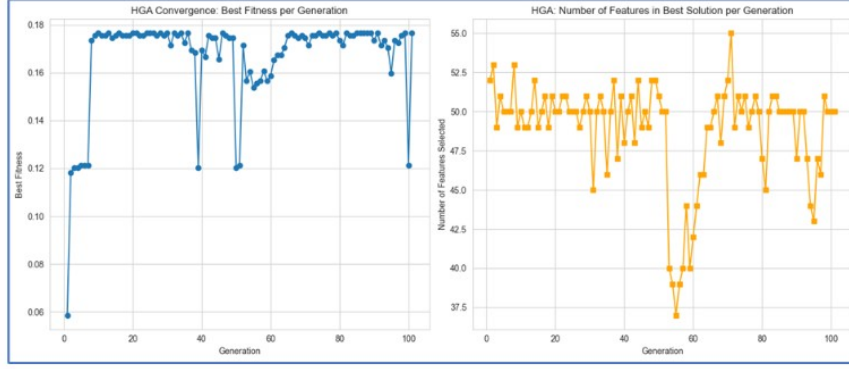


Figure 4: GA results for the fitness (left) and number of features (right) for an individual found after 100 generations on early experimentation

	Precision	Recall	F1-score	Support
0	0.9402	1.0000	0.9692	440
1	1.0000	0.0968	0.1765	31
Accuracy			0.9406	471
Macro avg	0.9701	0.5484	0.5728	471
Weighted avg	0.9441	0.9406	0.9170	471

Table 3: Report of performance metrics for a solution found through early experimentation

4.1.4 Results

As can be seen in Figure 4 and Table 3, the results were far from optimal; not only did the GA seem to get stuck on local “optima,” but in general, the fitness values were very low. Possible reasons: Low fitness due to the evaluation scores from the classifier is caused by the highly imbalanced nature of the dataset. The GA appears unable to come up with a proper combination of features in the intended number of generations. Likely, more data pruning is necessary to allow the GA to explore only potentially useful features, speeding up the process.

4.2 Final set experimentation

In this section, emphasis is placed on those aspects of the trials that were changed to the initial experimentation.

4.2.1 Data Preprocessing

In addition to what was done in the early experiments, in this instance, all the features with zero variance (i.e., with the same value in all samples) were removed from the dataset, resulting in 116 out of 590 features filtered, leaving 474 in the final dataset.

4.2.2 Splitting of the dataset for Fitness Evaluation

Considering the imbalanced nature of the dataset, the splitting was done with stratification to guarantee the representation of the minority class; again, 70% was assigned for training and 30% was kept for further validation of the classifier model. This resulted in 1,096 and 471 samples for training and validation, respectively.

4.2.3 Feature scaling and oversampling

Scaling:

Even though feature scaling does not significantly improve Random Forest classifier performance in general, a standard scaling was applied to the training and validation data separately. This was done by removing the mean value of each feature and scaling it to unit variance. The standard score of a sample x is calculated as:

$$z = \frac{x - u}{s}$$

where u is the mean and s is the standard deviation of the training samples.

Oversampling:

With the intention to improve the underlying model performance on the imbalanced classification task when the minority class is underrepresented, an oversampling of such class in the training dataset was implemented through the Synthetic Minority Over-sampling Technique (SMOTE). The result is shown in Figure 5.

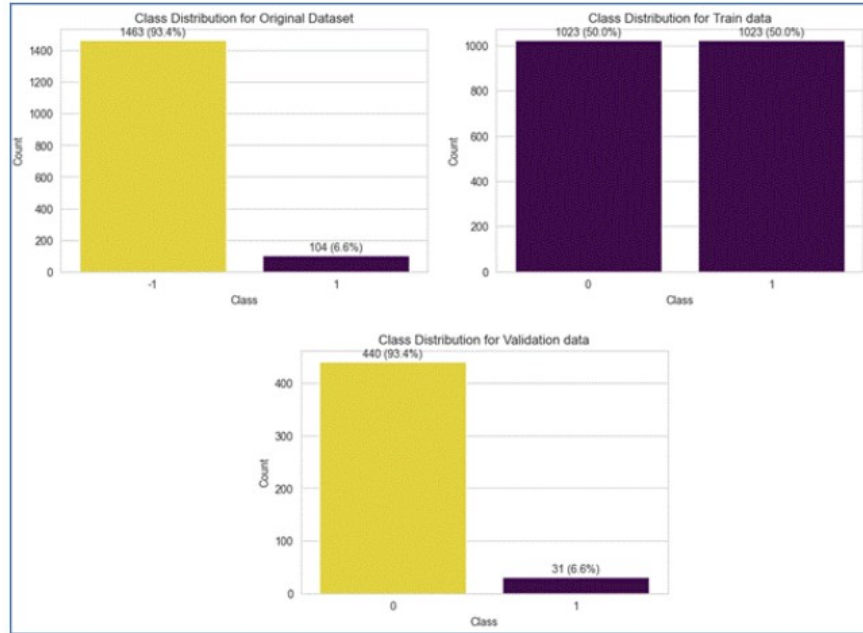


Figure 5: Resulting Class distribution on the datasets after applying SMOTE to the training dataset

4.2.4 Fitness function implementation

The goal of analyzing an industrial binary labeled dataset such as SECOM is to train a model that, given a set of feature values, classifies correctly within an acceptable uncertainty, the output of a manufacturing process as defective or not. Under this context, it can be argued that it is more important to correctly predict the “defective” class than the non-defective one; the problem to be solved is then considered both imbalanced and asymmetric. A decision threshold has been added as a hyperparameter to introduce this bias, applied at the evaluation of the probability predicted by the classifier, as seen in the equation.

$$P_r \geq 0.3.$$

Name	Value / Type	Description
Initial population	Random weighted: $p_i = d/D$	To make it more likely for the algorithm to focus on small subsets instead of larger ones in further generations.
Selection Mechanism	Rank-based with selection coefficient from 0.4 to 0.7	To select pairs of parents using rank-based roulette wheel selection
Genetic operation control	Generational	Updates the whole or most of the population with the offspring
Crossover	Uniform from 1 to 3 points. Rate = 1	
Mutation	Rate from 0.01 to 0.1	
Population Size	50	
Number of generations	100	
Penalty coefficient	From 0.001 to 0.1	Higher value means GA can explore farther from the desired features number.
Desired Features Number	50	Based on the benchmark
Decision Threshold	0.3	The minimum probability to consider a prediction a positive class.

Table 4: Experiment Hyperparameters values

4.2.5 Best Result Obtained

Experiment: 12, fitness: 0.5165

The values of the hyperparameters for this experiment are shown in Table 4, the results represent a significant improvement compared to the initial experiments, although they still fall short of optimal. During the early generations, the GA rapidly diverged from the initial population’s best solution, consistently discovering fitter individuals. As shown on the left

Number of features	Generation	Selection coeff.	Crossover points	Crossover rate	Mutation coeff.	Penalty coeff.
74	32	0.7	3	1	0.1	0.001

Table 5: Parameters of the best solution found

side of Figure 6, the fitness improved steadily until around generation 30, where it began to plateau, suggesting convergence to a local optimum. The number of features selected varied substantially across generations, initially ranging from around 50 to over 80, the feature count peaked around generation 30 before declining and stabilizing at approximately 55 features, as observed on the right side of Figure 6, specifically, the best fitness score achieved was 0.5165, with 74 features selected, found at generation 32 (see Table 5). This behavior indicates that the GA was able to explore and eventually exploit smaller yet effective feature subsets.

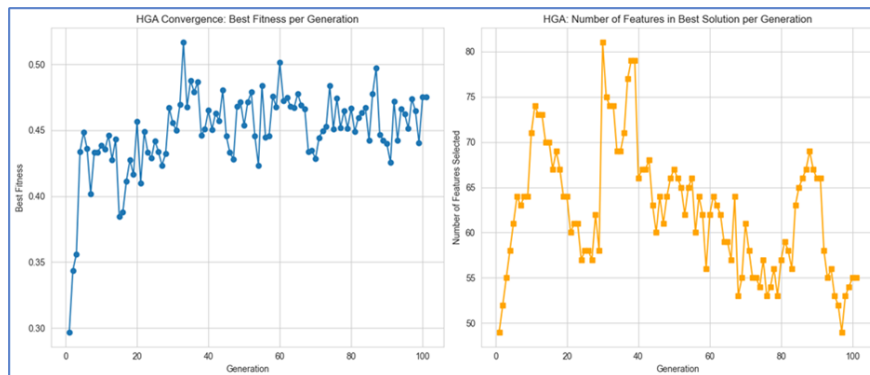


Figure 6: GA results for the fitness (left) and number of features (right) for the best individual found after 100 generations

	Precision	Recall	F1-score	Support
0	0.9743	0.9477	0.9608	440
1	0.4651	0.6452	0.5405	31
Accuracy			0.9278	471
Macro avg	0.7197	0.7964	0.7507	471
Weighted avg	0.9408	0.9278	0.9332	471

Table 6: Report for the Best solution found with fitness: 0.5165

4.2.6 Analysis

Around generation 30, the GA appeared to have found a promising evolutionary path, converging to a solution with a fitness of 0.5165. Although this fitness value did not improve significantly in subsequent generations, the classifier built on this solution demonstrated decent overall performance.

As detailed in the classification report in Table 6, the solution achieved an accuracy of 92.78%, however, as expected, the model showed an imbalance in performance between the two classes: For class 0, the majority class (440 samples), the F1-score was excellent at 0.9608. For class 1 (31 samples), we had a considerably lower F1-score of 0.5405, although given the class imbalance, this is still reasonable compared to the baseline (Ttest) in Table 1. That is, the best GA solution found with 75 features is correctly classified in approximately 54% of the positive cases (fails), compared to approximately 59% of the baseline example with 40 features. The macro-averaged F1-score was 0.7507, while the weighted average F1-score reached 0.9332, reflecting the classifier’s strong bias toward the majority class. This trade-off suggests that while the GA effectively selected relevant features, further improvements may require incorporating methods to better handle class imbalance, such as modified fitness functions, resampling techniques, or cost-sensitive learning. Despite continued evaluations, the algorithm did not manage to improve this fitness score in subsequent generations. This plateau in performance can be attributed to several factors.

Convergence to a Local Optimum:

The GA might have converged prematurely to a local optimum. The stabilization of fitness values around generation 30 suggests that the population became genetically similar, reducing further exploration of the search space.

Lack of Diversity:

A small population of 50 individuals and the high selection pressure ($q = 0.7$) may have led to a loss of diversity too early in the run. Without sufficient variation in the population, crossover and mutation operations might have been less effective, limiting the generation of new and potentially better solutions.

Ineffective Genetic Operators:

After discovering the promising path, the crossover and mutation operations may have failed to produce substantially different or improved individuals, indicating the need for more exploratory operators or adaptive mutation strategies.

Penalty Interference:

Given the fitness function’s penalty for deviating from the target number of features (e.g., 50), some potentially better-performing feature sets might have been too heavily penalized, discouraging the algorithm from exploring them further.

5 Experiments with the Synthetic Data

Manufacturing datasets in semiconductor production frequently exhibit data quality challenges, including missing sensor readings, measurement noise, and insufficient fault examples

for training robust classifiers. This data scarcity problem significantly impacts the performance of machine learning models, particularly in fault detection scenarios where failure cases are rare but critical to identify.

To address these data quality limitations, a comprehensive synthetic data generation method has been implemented for this project using statistical sampling techniques combined with domain knowledge from semiconductor manufacturing processes embedded in the SECOM dataset. This synthetic data generation approach enables the creation of additional testing examples while preserving the statistical properties and feature relationships inherent in real manufacturing data.

This synthetic data augmentation strategy allows us to create controlled experimental conditions where the optimal feature subsets are known a priori, enabling systematic validation of the genetic algorithm’s feature selection performance before application to real-world manufacturing data. The synthetic dataset generation methodology provides a crucial first-stage validation environment that bridges the gap between theoretical algorithm development and practical industrial implementation.

5.1 Synthetic dataset generation

The synthetic data generation process employs a multistage approach that first analyzes the statistical distribution of each parameter in the original SECOM dataset, then generates new samples using Gaussian mixture models calibrated to match the observed variance and correlation patterns. This statistical modeling technique ensures that synthetic fault scenarios maintain realistic sensor value ranges and inter-sensor dependencies typical of semiconductor manufacturing environments.

Step / Description	Mathematical Formulation
Estimated mean vector and covariance from data (1,567 × 591)	$\mu = \mathbb{E}[X], \Sigma = \text{Cov}(X)$
For each synthetic sample, draw standard normal noise vector	$\mathbf{z} \sim \mathcal{N}(0, I_{591})$
Generate synthetic data by affine transform	$\mathbf{X}_{\text{synth}} = \mu + L\mathbf{z}, LL^T = \Sigma, \mathbf{z} \sim \mathcal{N}(0, \Sigma)$

Table 7: Process for Generating Synthetic Data

Table 7 summarizes the key parameters used in the synthetic data generation process, including the number of synthetic samples generated for each fault category, the statistical parameters (mean, variance, correlation coefficients) derived from the original dataset, and the validation metrics used to ensure the quality of synthetic data.

5.2 Analysis

We applied GA model with same hyper parameters , same data pre/processing for both SECOM and Synthetic dataset. The convergence of the SECOM dataset (Figure 6) is slower and more variable, but achieves greater ultimate fitness, underscoring the richness and complexity of genuine sensor relationships. In contrast, the GA on synthetic data (Figure 7) converges faster and more stably - probably due to the smoother, lower complexity landscape of the Gaussian-modeled inputs - but cannot fully recover the performance level seen with real process measurements.

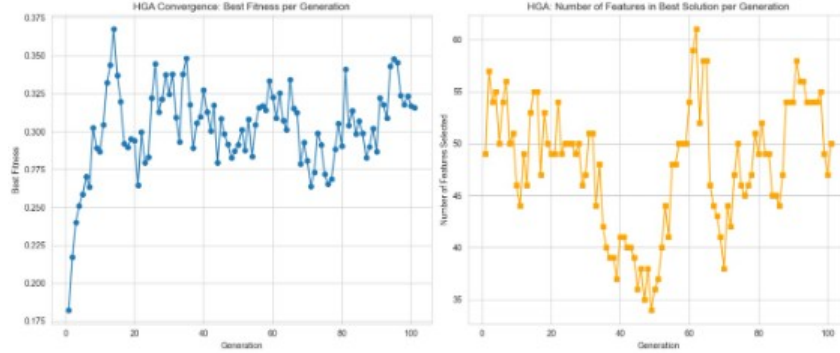


Figure 7: GA results for the fitness (left) and number of features (right) for the best individual found after 100 generations on Synthetic Data

	Precision	Recall	F1-score	Support
0	0.9599	0.8805	0.9185	435
1	0.6188	0.5556	0.3704	36
Accuracy			0.8556	471
Macro avg	0.6188	0.7180	0.6444	471
Weighted avg	0.9078	0.8556	0.8766	471

Table 8: Result of Synthetic Dataset

By comparing Table 6 and Table 8, the most significant performance gap emerged in minority class (fault) detection, where the real SECOM dataset achieved 54.05% F1-score compared to only 37.04% on synthetic data - a degradation of 17.01 percentage points. The synthetic dataset achieved higher precision for minority class detection (61.88% vs. 46.51%) but suffered substantial recall loss (55.56% vs. 64.52%). This suggests the synthetic data may have introduced artificial patterns that improved specificity at the cost of sensitivity.

6 Related Work

Feature selection in machine learning is necessary to reduce dimensionality without degrading the classification performance. The conventional methods, such as Sequential Forward Search (SFS) and Sequential Forward Floating Search (SFFS), have been contrasted with Genetic Algorithms (GAs), and it has been observed from research that GAs perform more effectively for large-scale problems because of their global search ability [3]. Standard GAs are augmented in [6] with local search operations (i.e., feature addition/removal) and a ripple factor to provide fine-tuning of solutions to mitigate weaknesses such as premature convergence. Their research experimentally verified on several datasets, shows that HGAs solve large problems more effectively than sequential approaches. Extending this, this project incorporates an adaptive ripple factor with population diversity weighting, parallel search for computational cost minimization, and multi-objective optimization for subset size vs. accuracy tradeoff [2, 7]. These enhancements are extensions of the framework presented in [6] but with the benefit of the lessons of previous research, e.g., Ferri et al.’s tradeoff analysis [5] and Kudo and Sklansky’s problem size guidelines [3]. Finally, the method in this project incorporates adaptive and parallelized mechanisms with steady-state GAs to improve the efficiency and scalability of feature selection, as demonstrated in practical applications such as fault detection [1].

7 Conclusions and Future Work

Conclusions from results

It should be noted that applying the same genetic algorithm with identical hyperparameters to two seemingly similar datasets yields significantly different results. For example, the F1-score on the SECOM dataset is 54.05%, while it drops to 37.04% on the synthetic dataset. Similarly, the recall is 64.52% for SECOM compared to 55.56% for the synthetic data. These discrepancies, if viewed without context, might suggest that one dataset is more imbalanced than the other. This raises the question whether the cross-validation procedure needs to be revised or whether the synthetic dataset produced in this project is an inadequate proxy to evaluate GA-based feature selection in the real-world manufacturing process data.

Algorithm improvements

As shown in the results of both datasets, the lack of diversity might be mainly attributed to the small number of individuals (50) in the population and a very strict penalty, to overcome these factors, future implementations should have larger populations and could incorporate techniques such as diversity-preserving strategies (e.g., fitness sharing, crowding), hybridization with local search, or adaptive genetic operators. Revisiting the fitness function to better balance accuracy and feature count may also enable further progress beyond current plateaus.

Alternative dataset

For large-scale experimentation, the Kaggle Bosch Production Line Performance dataset offers a challenging test setup with a higher number of features and observations than the UCI SECOM dataset. Its industrial-scale manufacturing sensor data offers a real-world setup to rigorously test the robustness, scalability, and noise resilience of the proposed feature selection algorithm. The inclusion of this dataset would enable rigorous testing with varying levels of dimensionality and complexity, more accurately simulating real-world industrial applications.

Other future work

The model trained on synthetic data could be tested on real data and vice versa as a way of testing the generality of the approach proposed on this project.

References

- [1] Heitor Nunes. Fault detection by feature selection: 590 to 44 [Internet]. Kaggle; [cited 2025 Jul 9]. Available from: <https://www.kaggle.com/code/heitornunes/fault-detection-by-feature-selection-590-to-44/notebook>
- [2] Deb K. Multi-objective optimization using evolutionary algorithms. Chichester: Wiley; 2001.
- [3] Guyon I, Elisseeff A. An introduction to variable and feature selection. J Mach Learn Res. 2003;3:1157–82.
- [4] UCI Machine Learning Repository. SECOM Data Set [Internet]. [cited 2025 Jul 9]. Available from: <https://archive.ics.uci.edu/dataset/179/secom>
- [5] IBM Corporation. Data mining with SPSS Modeler. Section 11.2.3.3.2: Identifying correlated features. Armonk (NY): IBM; [no date].
- [6] Oh IS, Lee JS, Moon BR. Hybrid genetic algorithms for feature selection. IEEE Trans Pattern Anal Mach Intell. 2004 Nov;26(11):1424–37.
- [7] Deb K. Multi-objective optimization using evolutionary algorithms. 1st ed. Chichester: Wiley; 2021.
- [8] Potharaju S, Tambe SN, Rao GM, Kantipudi MVVP, Bamane KD, Bendre M. Min3GISG: A synergistic feature selection framework for industrial control system security with the integrating genetic algorithm and filter methods. [Accepted 2025 Apr 8].

GitHub Repositories

HGA Feature Selection Repository